

Практическая работа 4

КОЛЛЕКЦИИ И ТИПОБЕЗОПАСНЫЙ СЧЁТЧИК

Цель работы

Реализовать обобщённую коллекцию для подсчёта повторов, безопасного объединения результатов и выдачи неизменяемого снимка.

Описание программы

`CountMap<T>` хранит число появлений каждого ключа в `HashMap`. Операции `add`, `remove`, `getCount`, `size` и `addAll` образуют минимальный контракт счётчика.

Общие сведения

Параметр типа `T` позволяет применять структуру к кодам продуктов и другим объектам без приведения типов. Объединение суммирует значения одинаковых ключей, не изменяя источник.

Функциональное назначение и ограничения

Счётчик подходит для агрегации повторяющихся продуктов или действий. Снимок нужен вызывающему коду для чтения состояния без возможности повредить внутреннюю карту.

`null`-ключи запрещены. Отсутствующий ключ имеет счётчик 0. `snapshot` возвращает самостоятельное неизменяемое представление; последующие изменения `CountMap` не отражаются в ранее полученном снимке.

Логическая структура

`add` использует `Map.merge(key, 1, Integer::sum)`. `addAll` обходит снимок источника и суммирует количества. `remove` удаляет ключ целиком. `snapshot` создаётся через `Map.copyOf`.

Используемые технические средства

Java Collections Framework, generics, `HashMap`, `Map.copyOf`, JUnit 5 и Gradle.

Входные и выходные данные

Входные данные. Ключи произвольного типа T и другой CountMap с совместимым типом для объединения.

Выходные данные. Количество конкретного ключа, число уникальных ключей и неизменяемая Map<T, Integer>.

Контрольные сценарии

Операция	Назначение	Гарантия
add	Увеличить счётчик	Типобезопасный ключ
addAll	Суммировать карты	Источник не меняется
snapshot	Отдать состояние	Копия неизменяема
remove	Удалить ключ	Остальные записи сохранены

Ход работы

Проект собран и проверен по командам из README и протокола проверки. Ниже приведены фактические фрагменты реализации и результаты запусков; учебные изображения преподавателя не использовались.

1. Реализация использует композицию над HashMap и скрывает изменяемую структуру. Обобщённые сигнатуры поддерживают безопасное копирование в карту надтипа.

```
1 package ru.sp.mju.collection;
2
3 import java.util.HashMap;
4 import java.util.Map;
5 import java.util.Objects;
6
7 /**
8  * Трёхкомпонентный счётчик попарных ключей.
9  * Ключом null не допускается, значение не является переопределяемым.
10  *
11  * Формат <K> - тип уникального ключа
12  */
13 public final class CountMap<K> {
14     private final Map<K, Integer> counts = new HashMap<>();
15
16     /** Считает текущий счётчик. */
17     public CountMap() {
18     }
19
20     /**
21      * Добавляет одно значение ключа.
22      *
23      * Формат value: ключ
24      * Формат: NullPointerException если value не String
25      * Формат: IllegalArgumentException если значением является null
26      */
27     public void add(K value) {
28         Integer requiredValue = Objects.requireNonNull(value, "Ключ должен быть не null");
29         counts.merge(requiredValue, 1, Math::addExact);
30     }
31
32     /**
33      * Возвращает число вхождений ключа или null, если ключ не встречается.
34      *
35      * Формат value: ключ
36      * Формат: IllegalArgumentException если value не String
37      * Формат: NullPointerException если value не String
38      */
39     public Integer get(K value) {
40         Integer requiredValue = Objects.requireNonNull(value, "Ключ должен быть не null");
41         return counts.get(requiredValue);
42     }
43
44     /**
45      * Возвращает сумму значений и возвращает его количество до удаления.
46      *
47      * Формат value: ключ
48      * Формат: IllegalArgumentException если value не String
49      * Формат: NullPointerException если value не String
50      */
51     public Integer remove(K value) {
52         Integer requiredValue = Objects.requireNonNull(value, "Ключ должен быть не null");
53         Integer previousCount = counts.get(requiredValue);
54         return previousCount == null ? 0 : previousCount;
55     }
56
57     /**
58      * Возвращает сумму значений ключей.
59      *
60      * Формат: Integer
61      */
62     public int size() {
63         return counts.size();
64     }
65
66     /**
67      * Преобразует все значения в другой тип, не изменяя источник.
68      *
69      * Формат: преобразователь значения типа (K, V) в тип, который принимает
70      * (CountMap<K> extends V).
71      *
72      * Формат: преобразователь значения
73      * Формат: NullPointerException если преобразователь не String
74      * Формат: IllegalArgumentException если преобразователь не String
75      */
76     public void addAll(CountMap<K> source, V target) {
77         CountMap<K> extends V requiredSource = Objects.requireNonNull(source, "Источники");
78         for (Map.Entry<K, Integer> entry : source.entrySet()) {
79             counts.merge(entry.getKey(), entry.getValue(), Math::addExact);
80         }
81     }
82
83     /**
84      * Возвращает полный неизменяемый список текущих значений.
85      *
86      * Формат: неизменяемый список на основе списка значений.
87      *
88      * Формат: неизменяемый список
89      */
90     public Map<K, Integer> toMap() {
91         return Map.copyOf(counts);
92     }
93
94     /**
95      * Возвращает количество в карте, которое принимает значение типа (K, V).
96      *
97      * Формат: преобразователь значения типа (K, V) в тип, который принимает
98      */
99 }
```

Рисунок 1 – Реализация обобщённого CountMap

2. Два набора объединены с суммированием FOOD-101. Попытка изменить snapshot отклонена, а ранее полученный снимок сохранил прежнее состояние.

```
Подсчёт и объединение

$ ./gradlew clean check run
Первый набор: {FOOD-101=2, FOOD-102=1}
После объединения: {FOOD-101=3, FOOD-102=1, FOOD-103=2}
Содержимое после сортировки совпадает: true
Источник объединения сохранён: true
Изменение снимка запрещено: true
Ранее полученный снимок не изменился: true
BUILD SUCCESSFUL
```

Рисунок 2 – Объединение и неизменяемый снимок

3. Двенадцать JUnit-тестов покрывают добавление, удаление, объединение, null, снимки и ковариантное чтение.

```
Проверки CountMap

CountMapTest
12 tests completed, 12 passed
Проверены add(), getCount(), remove(), addAll(), snapshot(),
нулевые значения, копирование в карту надтипа и стабильность снимка.
BUILD SUCCESSFUL
```

Рисунок 3 – Результаты тестов CountMap

Вывод

Создан универсальный CountMap с ясным контрактом. Он корректно считает и объединяет элементы, не раскрывает внутреннюю карту и предоставляет стабильные неизменяемые снимки.

Студент группы 090302-ИСТа-023

Карионов В.В.

Преподаватель

Елкин Н.С.